

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

Source: Github

Github Repository Link:

https://github.com/ssvibitha/DeepLearningLab_2026-27.git

7. Experimental Results

- This experiment implements the Perceptron algorithm to classify banknotes as authentic or forged using the Banknote Authentication dataset. The model was trained for 100 epochs and its performance was evaluated.
- The Perceptron achieved an accuracy of 98.55% on the test dataset.
- It obtained a precision of 96.83%, recall of 100%, and F1-score of 98.39%.
- The training error decreased rapidly during the initial epochs and then remained nearly stable.
- The confusion matrix showed that most samples were classified correctly, with only a few false positive predictions.
- The weights and bias converged to stable values after 100 epochs, indicating successful model training.

8. Generated Plots with Interpretation

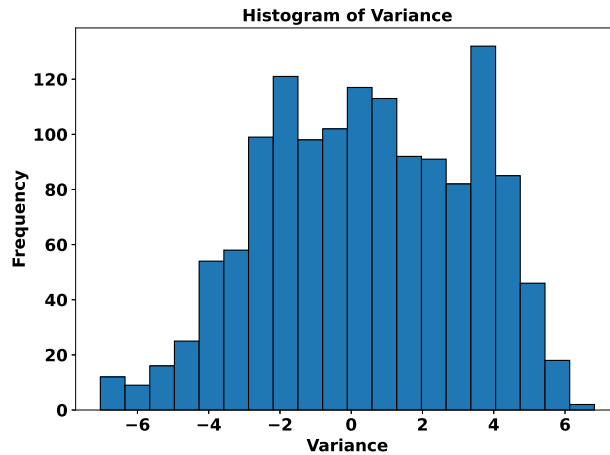


Figure 1: Histogram of Variance

Histogram of Variance Inference:

The histogram shows that the feature values are distributed over a wide range indicating good variability among the samples. Most observations are present in the middle region, while fewer samples are present at the extreme values. Overall the plot is well distributed and can contribute effectively to the classification of authentic and forged banknotes.

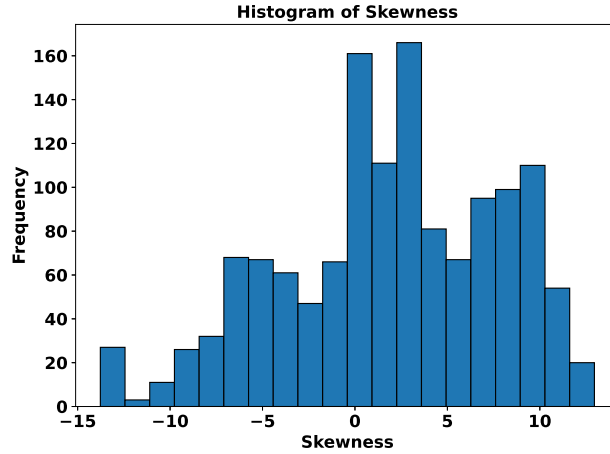


Figure 2: Histogram of Skewness

Histogram of Skewness Inference:

The skewness feature values are distributed across both positive and negative values. Most of the skewness values are concentrated on the values between 0 and 5. This shows that the some of the values have skewness close to 0 while the rest has moderate skewness. This variation helps perceptron to classify between authentic and forged banknotes.

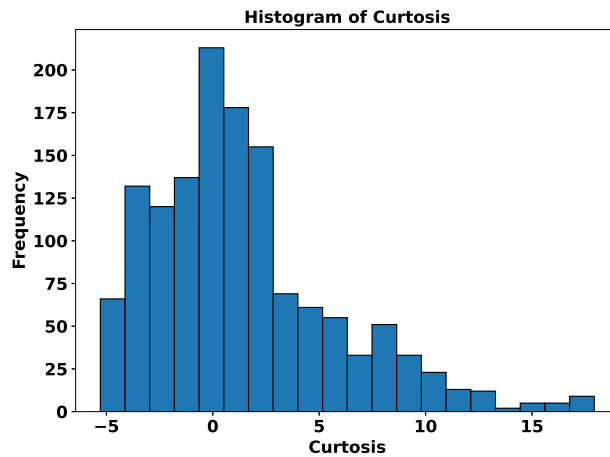


Figure 3: Histogram of Kurtosis

Histogram of Kurtosis Inference:

The histogram shows a right-skewed distribution with a few outliers towards the extreme right. The presence of a wide range of values helps distinguish authentic banknotes from forged ones.

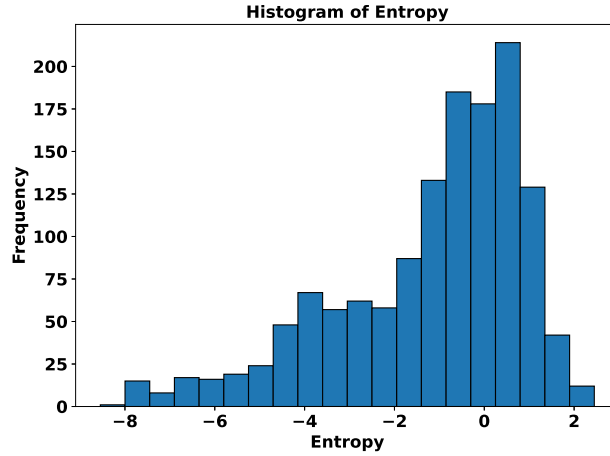


Figure 4: Histogram of Entropy

Histogram of Entropy Inference:

The histogram shows a left-skewed distribution with very few outliers on the extreme left of the histogram. Majority of the entropy values are concentrated in the range of -2 and 1. This variation provides helps the perceptron learn a decision boundary to distinguish between authentic and forged banknotes.

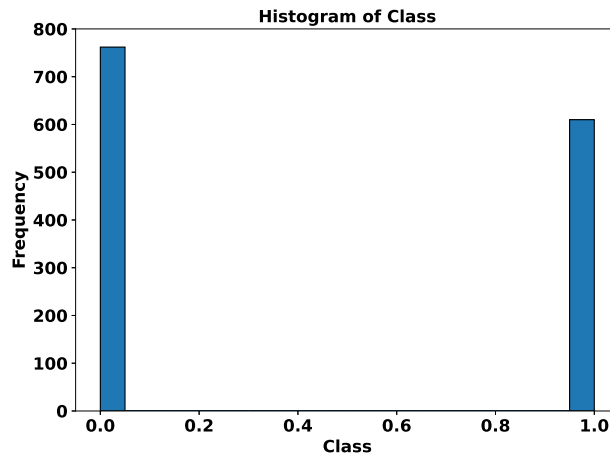


Figure 5: Histogram of Class

Histogram of Class Inference:

The histogram shows that the dataset contains samples from both classes in comparable proportions. This shows that the dataset is well balanced and helps the perceptron to learn both classes effectively.

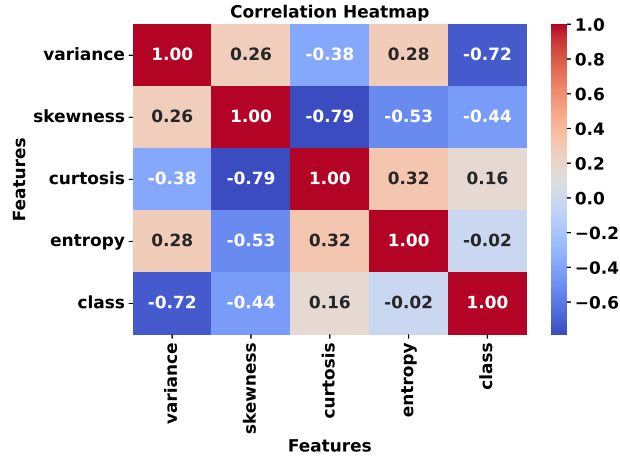


Figure 6: Correlation Heatmap

Correlation Heatmap Inference:

The heatmap shows that variance is the most important feature for classification as it has the strongest negative correlation with the class with a value of -0.72. Skewness and kurtosis are closely related, while entropy has a weak correlation with the class.

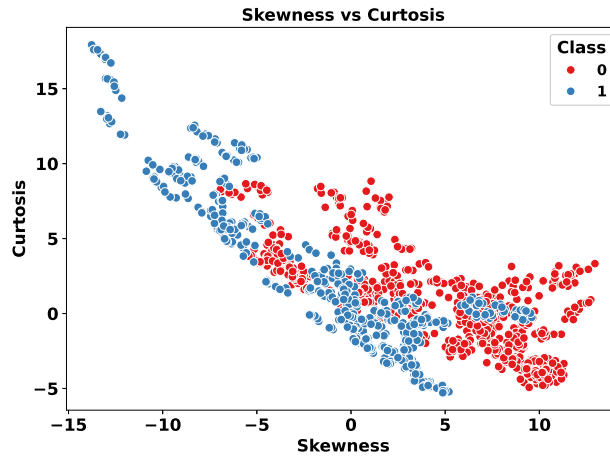


Figure 7: skewness vs kurtosis

Skewness vs Kurtosis Scatter Plot Inference:

The scatter plot shows a negative relationship between skewness and kurtosis. The two classes overlap in some regions, however these features together can assist the perceptron in separating authentic and forged banknotes.

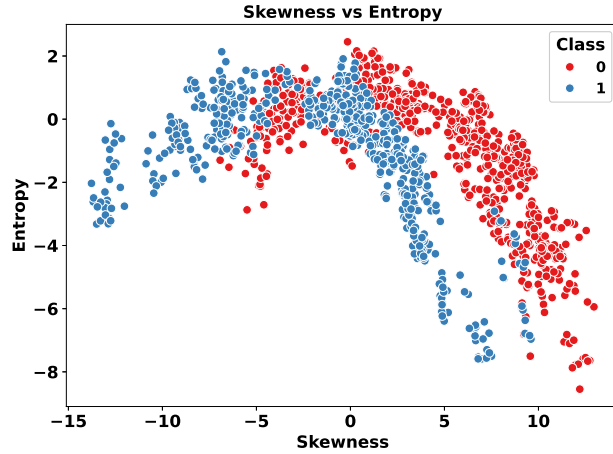


Figure 8: skewness vs entropy

Skewness vs Entropy Scatter Plot Inference:

The two classes Skewness and Entropy are overlapping in the middle region. This shows that a non-linear classifier is required to separate these classes.

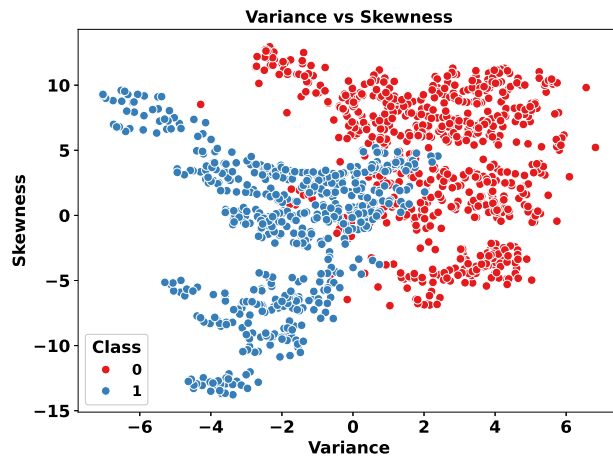


Figure 9: variance vs skewness

Variance vs Skewness Inference:

In this plot Skewness and Variance help distinguish the two classes well despite having a few overlaps in the middle. The distribution of these points suggests that these two features can help separate authentic and forged banknotes.

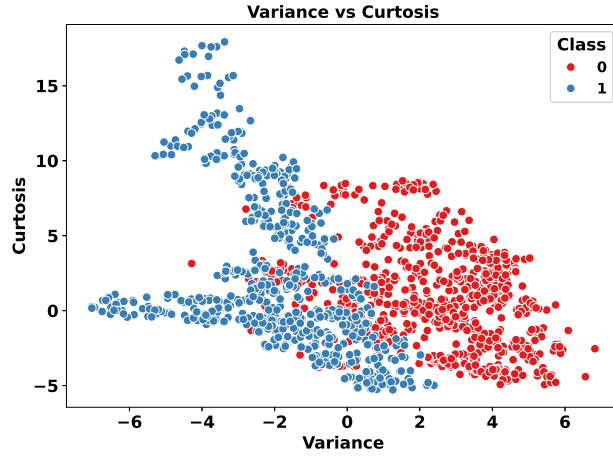


Figure 10: variance vs curtosis

Variance vs Cusrtnosis Inference:

In this plot, the distribution of samples shows a clear separation between the two classes. The slight overlap between the clusters indicates that these features are useful for classification.

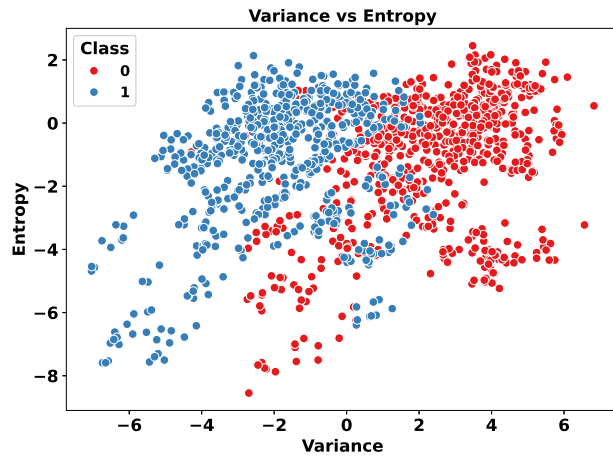


Figure 11: variance vs entropy

Variance vs Entropy Inference:

The scatter plot shows moderate overlapping between the two classes. This shows that Entropy and Variance together helps to distinguish between the two classes effectively.

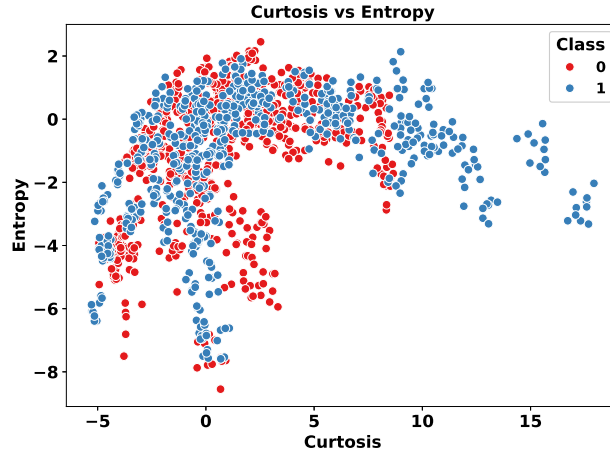


Figure 12: curtosis vs entropy

Curtosis vs Entropy Inference:

The scatter plot shows significant overlap between entropy and curtosis. Non-linear decision boundaries are required to separate the classes.

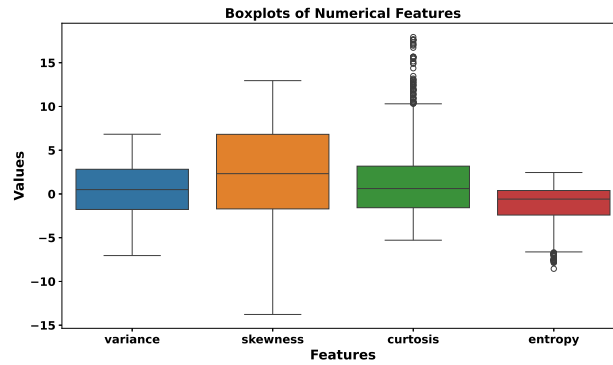


Figure 13: Boxplots Of All Features

Boxplots Inference:

The box plot shows that curtosis contains the most outliers followed by entropy. The feature values of Variance are well distributed, whereas the feature values of skewness follow a wide range of values.

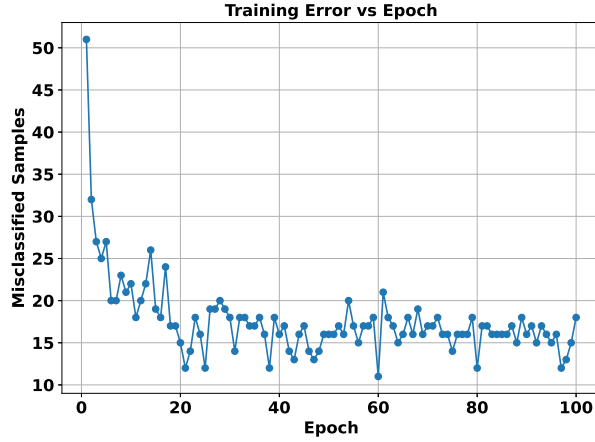


Figure 14: Training Error vs Epoch

Training error vs Epochs Inference:

From the plot, we can infer that the model learns quickly at first, then the error fluctuates slightly until the 100th epoch. This shows that the error decreases rapidly at the beginning and then reaches a stable level.

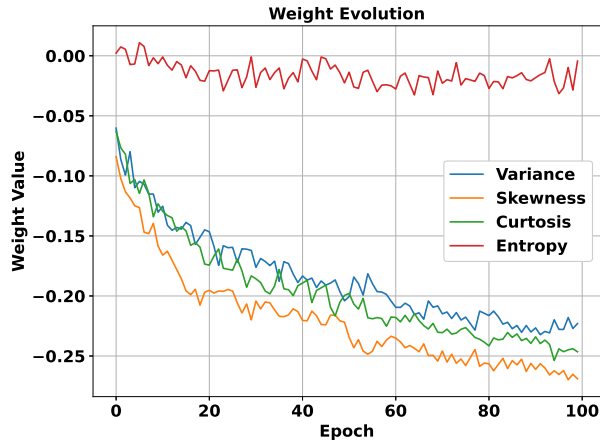


Figure 15: Weight Evolution

Weight Evolution Inference:

From the plot we can infer that the features Skewness, Kurtosis and Variance receive increasing negative weights. Entropy receives weight updates almost close to zero.

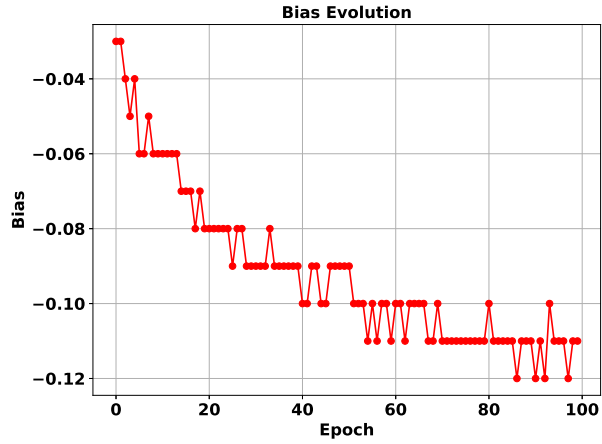


Figure 16: Bias Evolution

Inference:

The bias changes steadily during training and becomes stable by the end of 100 epochs. This shows that the model gradually converges to a stable solution.

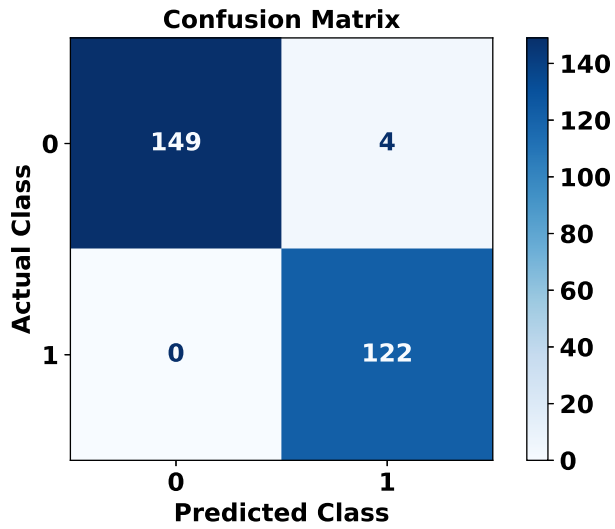


Figure 17: Confusion Matrix

Inference:

The confusion matrix shows that the model classifies most samples correctly with very few errors. It correctly identifies all Class 1 samples and makes only a few false positive predictions.

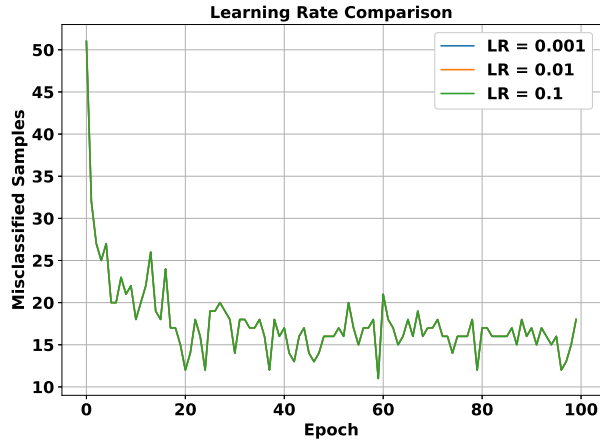


Figure 18: Learning Rate Comparison

Inference:

All three learning rates produce almost identical error curves during training. This suggests that the learning rate has minimal impact on the model's convergence.

9. Performance Tables

Training Summary

Dataset Size	1372x5
Train/Test Split	80:20
Learning Rate	0.01
Epochs	100
Final Weights	[-0.22298399 -0.26903532 -0.24649669 - 0.00438611]
Final Bias	-0.10999999999999999
Accuracy	0.9855
Precision	0.9683
Recall	1.0000
F1-score	0.9839

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	51	-0.0602	-0.0841	-0.0630	0.0023	-0.03
2	32	-0.0863	-0.1022	-0.0766	0.0074	-0.03
3	27	-0.0993	-0.1133	-0.0820	0.0055	-0.04
4	25	-0.0799	-0.1184	-0.1063	-0.0073	-0.05
5	27	-0.1099	-0.1249	-0.1031	-0.0068	-0.04
6	20	-0.1045	-0.1264	-0.1146	0.0109	-0.06
7	20	-0.1070	-0.1469	-0.1033	0.0077	-0.06
8	23	-0.1152	-0.1481	-0.1144	-0.0082	-0.05
9	21	-0.1152	-0.1396	-0.1341	-0.0017	-0.06
10	22	-0.1303	-0.1585	-0.1233	-0.0066	-0.06
...						
96	16	-0.2278	-0.2654	-0.2440	-0.0316	-0.11
97	12	-0.2279	-0.2621	-0.2464	-0.0267	-0.11
98	13	-0.2180	-0.2698	-0.2449	-0.0098	-0.12
99	15	-0.2271	-0.2651	-0.2440	-0.0286	-0.11
100	18	-0.2230	-0.2690	-0.2465	-0.0044	-0.11

Table 1: Epoch-wise learning of the proposed Single Layer Perceptron.

10. Discussion

- **Step function, Sigmoid function comparison**

The Step Activation Function produces only binary outputs (0 or 1) based on a threshold, whereas the Sigmoid Activation Function produces continuous values between 0 and 1. The Step function is not differentiable at the threshold and has a zero gradient elsewhere, making it unsuitable for gradient based optimization methods such as backpropagation. However, the Sigmoid function is smooth and differentiable, allowing efficient weight updates during training. Therefore, deep learning models use differentiable activation functions such as Sigmoid.

- **Implementation with Scikit-learn’s Perceptron.**

The perceptron implemented from scratch provides a clear understanding of the perceptron learning algorithm by manually performing weight initialization, forward propagation, and weight updates. Whereas Scikit-learn’s Perceptron is a highly optimized implementation that includes additional features such as parameter tuning, regularization, and efficient optimization. Although both implementations produced similar classification performance on the Banknote Authentication dataset, the Scikit-learn model was more efficient and convenient for practical applications.

- **Performing the experiment with different learning rate(0.001, 0.01, 0.1)**

The training error curves for all three learning rates almost completely overlapped, indicating

that the perceptron exhibited similar convergence behavior in each case. This suggests that, for the normalized Banknote Authentication dataset, the learning rate had minimal impact on the convergence pattern and classification performance within the chosen range. Therefore, all three learning rates were found to be suitable for training the perceptron on this dataset.

- **Why a Single Layer Perceptron cannot solve the XOR problem.**

A Single Layer Perceptron can only learn linearly separable patterns by constructing a single linear decision boundary. The XOR problem is not linearly separable because no single straight line can correctly separate its output classes. As a result, a Single Layer Perceptron fails to classify XOR inputs correctly. Solving the XOR problem requires a Multilayer Perceptron (MLP) with one or more hidden layers and nonlinear activation functions.

- **Effect of feature normalization on model convergence**

Feature normalization scales all input features to a similar range before training, preventing features with larger numerical values from dominating the learning process. This results in more stable weight updates, faster convergence, and improved training efficiency. In this experiment, feature normalization helped the perceptron learn a better decision boundary and contributed to achieving higher classification performance compared to training with unnormalized features.

11. Conclusion

This experiment successfully demonstrated the implementation of a Single Layer Perceptron for binary classification using the Banknote Authentication dataset.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.